



UNIVERSITY OF
LEICESTER

School of Computing and Mathematical Sciences
CO7201 Individual Project

Preliminary Report

**HomifyOne: An AI-Powered Personalisation
and Workflow Management Platform for
New-Build Residential Developments**

Student Name: Mrudhulaa Peechanary Vinodkumar

Student Email Id: mpv4@student.le.ac.uk

Student Number: 259025897

Project Supervisor: Dr Newman Lau

Principal Marker: Dr Zedong Zheng

Word Count: 6093

Date: 25/06/2026

DECLARATION

All sentences or passages quoted in this report, or computer code of any form whatsoever used and/or submitted at any stages, which are taken from other people's work have been specifically acknowledged by clear citation of the source, specifying author, work, date and page(s). Any part of my own written work, or software coding, which is substantially based upon other people's work, is duly accompanied by clear citation of the source, specifying author, work, date and page(s). I understand that failure to do this amounts to plagiarism and will be considered grounds for failure in this module and the degree examination as a whole.

Name: Mrudhulaa Peechanary Vinodkumar

Date: 25/06/2026

Contents

Chapter 1. Aims and Objectives	4
1.1. Project Motivation and Context	4
1.2. Target users and their problems	4
1.3. Aim	5
1.4. Proposed Solution Overview	5
1.5. Technical Challenges	5
Chapter 2. Requirements.....	7
2.1. Essential Requirements	7
2.2. Desirable Requirements	8
2.3. Optional Requirements	9
Chapter 3. Technical Specification	10
3.1. Tech Stack Overview	10
3.2. System Design Architecture	11
3.3. Backend architecture.....	11
3.3.1. Middleware Stack	12
3.3.2. Backend Endpoints	12
3.4. Frontend architecture	13
Chapter 4. Requirements Evaluation Plan	15
4.1. Evaluation Criteria	15
4.2. Evaluation Participants	15
4.3. Testing.....	15
4.4. Verification	15
Chapter 5. Background Research and Reading list.....	16
5.1. Existing Platform Gap Analysis.....	16
5.2. Research Element 1: NLP-Based Semantic Recommendation Engine	17
5.3. Research Element 2: Conversational AI Assistant	17
5.4. Theoretical Foundation for Lifestyle and Behavioural Preference Modelling	18
5.5. Reading List	19
Chapter 6. Time-plan and Risk Plan	20
6.1. Time-plan (Gantt chart)	20
6.2. Detailed Timetable.....	20
6.3. Milestones	22
6.4. Risk plan	22
Chapter 7. References	24

Chapter 1. Aims and Objectives

1.1. Project Motivation and Context

The UK new-build housing is seen as a sector where the largest amount of financial commitment is made by buyers. Research shows that UK housebuilders have not put the consumer at the centre of the new-build experience [1]. The Commission for Architecture and the Built Environment (CABE) found that home buyers' opinions have a limited impact on what is offered to them [2]. One of the studies by Competition and Markets Authority (2024) has found that buyers often cannot make well-informed decisions at important points during the transaction process [3]. Studies in construction management confirm that buyers place a high value on the ability to select the choices and extras for their homes according to their preferences, yet the industry lacks the ability to capture and respond to those preferences on a large scale [4]. Despite the gradual digitisation efforts, the home personalisation arena is still fragmented. Existing platforms serve only one or two stakeholder groups at most. There has not been a commercial tool so far that brings Buyer, Developer, Supplier, and Administrator workflows into a single application nor one available for real-time multi-stakeholder collaboration in this domain [5], [6]. Thus, HomifyOne is introduced to fix these gaps.

1.2. Target users and their problems

HomifyOne caters to four different stakeholders facing unique and unaddressed problems.

- Firstly, the buyers, especially the first-time buyers may not have proper knowledge about choosing the right choices and paid extras for their new home. They have to select products from very large catalogues for which they do not get proper guidance. They may also not be aware of their budget or which products are suitable for their lifestyle. Since buying a home is a one-time permanent investment, buyers should get intelligent support based on their needs and preferences [4], [7].
- Secondly, the developers who often manage multiple buyers simultaneously also must handle many other chores which includes coordinating selection approvals, purchase orders, and supplier communication [8], [9]. However, in many cases, this work is split among email, spreadsheet and general project management tools which can make their work tedious and time-consuming.
- Thirdly, the suppliers receive unstructured order communication from the developer but have no dedicated portal for accepting orders, confirming their delivery timeline, and submitting invoices.
- Lastly, the administrator does not have a centralised governance layer to manage user accounts, maintain product data, and monitor platform activities in one place. Without such a system, it becomes difficult to control users, update product information, and oversee the smooth functioning of the platform.

1.3. Aim

The main aim of this project is to design, build and evaluate HomifyOne as an efficient web application that replaces the current disjointed new-build personalisation and workflow process by offering a unified, intelligent digital experience for Buyers, Developers, Suppliers and Administrators.

1.4. Proposed Solution Overview

HomifyOne is a centralised, role-based web application that brings all four stakeholders into one platform. Rather than giving the buyers a catalogue to browse alone, it captures their personal preferences via a questionnaire and then uses AI-driven semantic matching to fetch the recommendations that fit to how the buyer intends to live in their home. A dynamic live pricing engine tracks how the buyer spends against their extras allowance in real-time. Besides that, HomifyOne manages the complete workflow process including developer review and approval, automated purchase order (PO) generation grouped by supplier, a dedicated portal for different suppliers, invoice management, and in-app notifications for all stakeholders at every workflow event. In addition to these, HomifyOne supports real-time collaboration among the users and also integrates various external APIs to provide the best experience for its users. It also includes AI capabilities to provide more personalised guidance and help buyers in making better decisions during the home personalisation process.

1.5. Technical Challenges

Delivering HomifyOne presents the following key technical challenges:

1. **NLP-driven recommendation pipeline:** I have not previously worked with NLP or semantic similarity systems. Implementing the recommendation engine requires me to research how sentence-transformer models such as all-MiniLM-L6-v2 convert text into vector representations, how FAISS performs cosine similarity search over those vectors, and how buyer's responses can be mapped to ranked extras recommendations per room. The challenge is not just associated with the technical integration but to understand how to create product and query embeddings to generate recommendations based on buyer's lifestyle preferences.
2. **Designing a Multi-Role Event-Driven Workflow:** Designing a system where one user action triggers a series of state transitions, automated document generation, and notifications across different roles is challenging. To address this, I'll analyze the use of event-driven architectural patterns, design a state machine diagram, and implement structured error handling for the smooth functioning of the system.
3. **No Prior Experience with Conversational AI:** Building an AI assistant is a new area for me, and I need to research to select the right approach. I will compare a lightweight embedding-based method using Sentence-Transformers and FAISS with a full RAG pipeline using the Gemini API, and choose the option that is best and feasible.

4. **Security and Role-Based Access Control:** Enforcing strict data boundaries across four user roles while maintaining a smooth user experience is a security challenge. I will implement RBAC with JWT authentication and bcrypt password hashing, apply middleware-level role validation on every API endpoint, and conduct access control testing to verify that users cannot access another role's data.

Chapter 2. Requirements

2.1. Essential Requirements

- Multi-role secure authentication for Buyer, Developer, Supplier, and Admin using JWT and bcrypt password hashing - Users log in through a single interface. Passwords are hashed with bcrypt before storage and a signed JWT is issued on login, carrying the user's role and ID to authenticate every subsequent API request.
- Role based access control and dashboards - Each role has a dedicated dashboard showing only the data and actions relevant to them. Access restrictions are enforced at the API level.
- Admin management of user accounts and product catalogue - Admins can create, edit and delete user accounts across all roles. They also manage the master product catalogue, including adding and updating products.
- Buyer personalised plot dashboard showing property details, selection status and deadline countdown - Each buyer sees their plot details, property address, and an overview of their selection progress across all rooms. A countdown timer displays the number of days remaining before their selection deadline.
- Room-by-room choices portal to browse and select standard choices for Buyers - Buyers navigate through their property room by room, browsing and selecting standard choices available for their specific plot type. Selections can be added, swapped, or removed at any point before submission.
- Buyer lifestyle questionnaire and AI recommendation engine for extras selection - Buyers complete a short questionnaire covering their lifestyle preferences. Responses are processed through sentence-transformer embeddings and FAISS semantic search pipeline to return a ranked list of personalised extras recommendations.
- Dynamic live pricing engine that recalculates the total extras cost as selections are added, removed or changed - As buyers add, remove, or change extras selections, the total cost recalculates instantly. The running total is displayed persistently on the buyer dashboard and alerts the buyer if their selections exceed their allocated budget.
- Developer sets deadline for Buyer selection, reviews, approves/rejects Buyer's submission - Developers assign a selection deadline to each buyer. Once a buyer submits, the developer reviews and either approves it which in turn trigger purchase order generation or rejects it with a written reason for revision.
- Automated purchase order generation and grouping by supplier - On approval, the system automatically generates purchase orders, consolidating all selected items from the same supplier into a single PO.

- Supplier restricted PO portal where they can view, acknowledge POs, update status and set ETA - Suppliers see only the purchase orders assigned to them. They can acknowledge receipt, update fulfilment status, and set an estimated delivery date.
- Supplier invoice upload against assigned purchase orders - Suppliers upload invoices directly against a specific PO within the platform, which keeps each invoice linked to its corresponding order for direct developer review.
- Developer PO and invoice management dashboard - Developers have a consolidated view of all purchase orders and supplier invoices. They can monitor fulfilment status, review uploaded invoices, and manage them as required.
- In-app notifications for key workflow events - Real-time notifications are delivered when important actions occur i.e., when a buyer submits selections, a developer approves or rejects, or when a supplier acknowledges a PO.

2.2. Desirable Requirements

- Real-time in-app messaging for relevant user communication using WebSocket connections via Socket.io - Buyers and developers can message each other directly within the platform. Conversations are persisted in the database and use the existing Socket.io infrastructure with dedicated chat rooms per Buyer-Developer pair.
- Calendar synchronisation for scheduled Buyer-Developer meetings and Buyer selection deadlines - When a developer sets a selection deadline or schedules a meeting, the platform creates corresponding events in both parties' Google Calendars via the Google Calendar API.
- Supplier ETA synchronisation to Developer's Calendar - When a supplier sets or updates a delivery date on a PO, the platform automatically syncs the date as a calendar event in the developer's Google Calendar.
- Developer interactive map displaying all assigned development location pins with a drill-down panel listing plots, buyers, and their selection status - Developers managing multiple sites can view all developments as location pins on an interactive map. Clicking a pin opens a side panel listing all plots, assigned buyers, and their current selection status.
- Standard choices comparator for Buyers - Buyers can select two or more standard choices within the same category, view them side by side and comparing their attributes to support more informed decision
- AI Buyer assistant for product selection and conversational guidance - An embedded conversational assistant lets buyers ask natural language questions about catalogue products. This will include a set of suggested questions that buyers can select to quickly ask common queries

- PDF selection summary generated on approval, downloadable by Buyer and Developer - When a submission is approved, the platform generates a PDF of the buyer's complete selections. This is downloadable by both buyer and developer.

2.3. Optional Requirements

- AI invoice summariser to extract key information from uploaded supplier invoices - When a supplier uploads an invoice PDF, the platform extracts key fields using an AI pipeline which in further reduces manual data entry for developers.
- AI warning system for missing choices and inconsistent selections for Buyers - The platform monitors buyer selections in progress and flags incomplete rooms, incompatible product combinations, and budget overruns before submission, giving buyers the chance to resolve issues in advance.
- Supplier invoice history and payment status dashboard showing all submitted invoices with paid/pending status updated by the Developer - Suppliers can view all invoices they have submitted along with the payment status updated by the developers
- Admin analytics dashboard providing platform-wide visibility into buyer progress, product trends, supplier performance through real-time data visualisations - Admins have access to real-time charts and KPI cards showing buyer completion rates, most selected products, supplier fulfilment performance, and developer approval activity across the entire platform.
- Email notifications for all key workflow events as a fallback to in-app notifications for all user roles - Along with in-app notifications, the platform sends email notifications for all key workflow events, thus making sure that users are informed even when they are not actively logged into the platform.

Chapter 3. Technical Specification

3.1. Tech Stack Overview

Category	Technology	Purpose
Frontend	React	JavaScript library for building client-side user interface
	HTML 5	Structure the content and layout of web page
	CSS3	Style and format web pages
	Tailwind CSS	Utility-first style framework for responsive UI development
	Redux Toolkit / Context API	Manage application state across components
	Axios	Handle HTTP requests from the frontend to backend
Backend	Node.js	Run JavaScript code on the server side
	Express.js	Create RESTful APIs and handle server-side routing
AI Microservice	Python	Programming language for AI service logic
	FastAPI	Build fast and lightweight API services in python
Database & ODM	MongoDB Atlas	Store all application data in a flexible, cloud-hosted document database
	Mongoose	Define schemas and interact with MongoDB from the Node.js backend
Authentication	JWT (JSON Web Tokens) & bcrypt	Manage secure authentication and password protection
Real-time features	Socket.io	Real-time communication between users
Media File Storage	Cloudinary	Store, manage and retrieve media files in the cloud
Email	Nodemailer & SendGrid	Send OTP password reset emails and event notification emails
PDF Generation	Puppeteer (Node.js)	Generate server-side PDF selection summaries upon developer approval
Third-party integration	Leaflet.js	Display interactive maps in the web application
	Google Calendar API / ical-generator	Create, manage and sync calendar events

AI-Embeddings	sentence-transformers (all-MiniLM-L6-v2)	Convert text into semantic vector representations for similarity matching
Vector Search	FAISS	Perform fast cosine similarity search over product embeddings
LLM (optional)	Google Gemini API	For generative AI or conversational AI capabilities
API Validation	Postman	Manually test and validate all API endpoints
Task Scheduling	nodecron	Schedule automated background tasks
Testing Frameworks	Jest & Supertest	Frameworks to test JavaScript code and backend API endpoints
Version Control	GitLab	Manage version control
IDE	Visual Studio Code	Code editor

3.2. System Design Architecture

The following diagram represents a high-level system design architecture of HomifyOne,

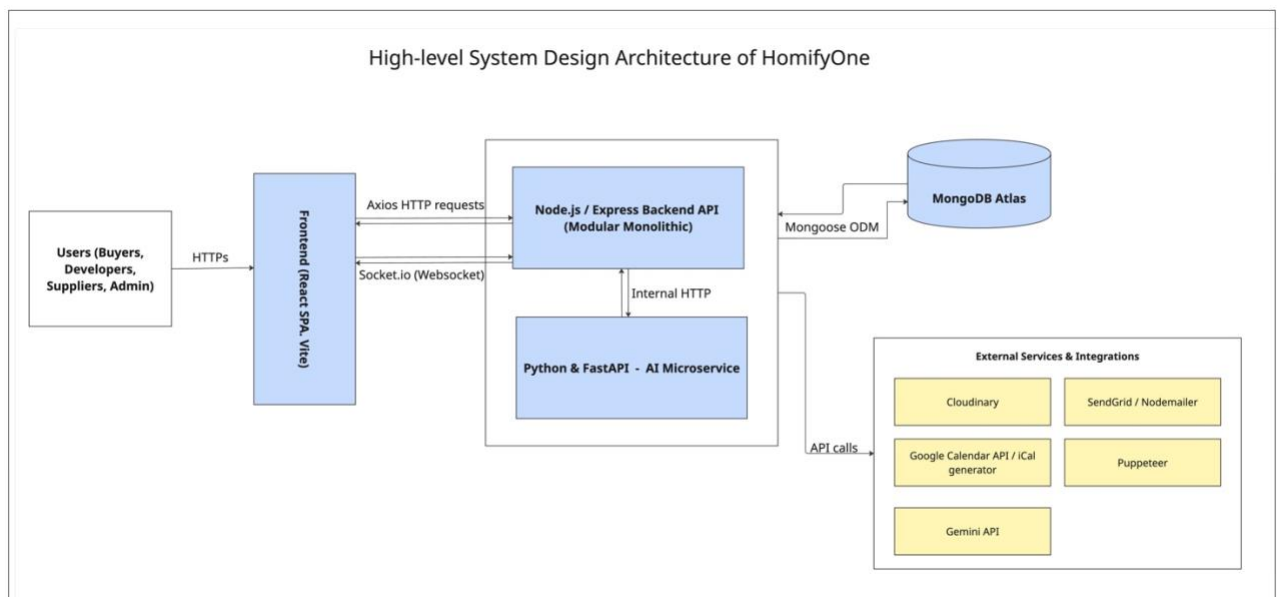


Figure 1: High-level System Design Architecture

3.3. Backend architecture

The backend is built as a modular monolithic application using Node.js and Express.js, internally structured using a layered MVC-style approach. Every incoming request is processed through shared middleware before reaching its route handler. For AI operations,

the Node.js backend delegates to a separate Python microservice built with FastAPI, via internal HTTP. This separation keeps the AI logic and its dependencies isolated from the main application server. This way, the Node.js layer remains lightweight and the two services can be scaled or updated independently.

3.3.1. Middleware Stack

Middleware	Purpose
helmet	Sets secure HTTP response headers
cors	Restricts cross-origin requests to the frontend origin
express.json()	Parses incoming JSON request bodies
morgan	HTTP request logging in development
verifyToken	Validates the JWT from the Authorization header on every protected route
express-validator	Validates and sanitises request body fields per route
errorHandler	Centralised error response formatter

3.3.2. Backend Endpoints

Method	Endpoint	Purpose
/api/auth		
POST	/login	Authenticate and return signed JWT
POST	/logout	Clear token from cookies
POST	/forgot-password	Send password reset OTP via email
POST	/reset-password	Validate OTP and update password
/api/buyer		
GET	/dashboard	Fetch plot details, selection status, deadline
POST	/questionnaire	Submit lifestyle questionnaire responses
GET	/recommendations	Retrieve AI-ranked extras recommendations
GET	/pricing	Get live recalculated extras total
POST	/submit	Submit full selection for developer review
GET	/summary	Retrieve approved selection summary & PDF link
/api/developer		
GET	/dashboard	Fetch all plots, buyers, and their statuses
GET	/buyers	List all buyers assigned to the developer
GET	/buyers/:id/submission	View a specific buyer's submitted selections

POST	/buyers/:id/approve	Approve buyer submission and trigger PO generation
POST	/buyers/:id/reject	Reject submission with written reason
PUT	/buyers/:id/deadline	Set or update buyer selection deadline
GET	/purchase-orders	List all generated purchase orders
GET	/purchase-orders/:id	View a specific PO in detail
GET	/invoices	List all supplier invoices
PUT	/invoices/:id/status	Mark invoice as paid or flagged
GET	/map	Fetch all development locations with plot and buyer data
/api/supplier		
GET	/purchase-orders	List all POs assigned to this supplier
GET	/purchase-orders/:id	View a specific PO in detail
PUT	/purchase-orders/:id/acknowledge	Acknowledge receipt of a PO
PUT	/purchase-orders/:id/status	Update PO fulfilment status
PUT	/purchase-orders/:id/eta	Set or update estimated delivery date
POST	/purchase-orders/:id/invoice	Upload an invoice against a PO
GET	/invoices	View all submitted invoices with payment status
/api/admin		
GET	/users	List all users across all roles
PUT	/users/:id	Update user details or role
GET	/catalogue	View full product catalogue
POST	/catalogue	Add a new product
GET	/analytics	Fetch platform-wide analytics data
Python Microservice layer - /api/ai		
POST	/recommend	Accept questionnaire responses and return ranked product recommendations
POST	/chat	Accept buyer message and return response
POST	/summarise-invoice	Accept invoice PDF text and return extracted key fields

Note: All of these may be subject to change as the project progresses.

3.4. Frontend architecture

The frontend will be developed as a single-page-application using React, with Vite used as the build tool. After a user logs in, the system will read their role from the JWT and redirect them to the correct dashboard using React Router DOM. Axios will be used to handle backend communication. A shared Axios instance will also be used such that the JWT Authorization header is included in every API request.

For real-time features, the frontend will use a Socket.io client connection. This connection will be created when the user logs in and closed when the user logs out. Redux Toolkit or Context API will be used to manage data and states while local component states will handle

smaller UI related data. Tailwind CSS will be used across the frontend to create a responsive layout and utility-based styling.

Chapter 4. Requirements Evaluation Plan

4.1. Evaluation Criteria

HomifyOne will be evaluated across functionality, security, performance and usability criteria. Functionality confirms that all features behave correctly under normal and edge-cases. Usability assesses whether each stakeholder can navigate their portal and complete tasks without technical guidance. Security verifies that role boundaries, JWT authentication, bcrypt hashing, and OTP validation are enforced correctly. Performance confirms acceptable response times and system stability throughout the usage.

4.2. Evaluation Participants

I will mainly evaluate the project through self-directed testing, which I will carry out after completing each feature. Thus, every role-specific workflow is executed properly and that issues are caught during development rather than at a later phase. Supervisor feedback will also serve as an external perspective on whether the platform behaves as expected.

4.3. Testing

The application will be tested in four stages. Firstly, I plan to do unit testing using Jest and Supertest to test backend functions and API endpoints. Secondly through integration testing, I will check whether the frontend, backend and AI microservice work together correctly. Thirdly, I will do API testing using Postman to manually test each API endpoints. Finally, I will apply system testing which will check the application's behaviour end-to-end ensuring that it behaves correctly as an integrated whole.

4.4. Verification

The project's success depends on how well it meets the requirements, supports functionality and usability, passes planned testing and remains stable in different user scenarios. Each requirement from the essential and desirable category will be tested against clear acceptance criteria. Optional ones will be noted as implemented or deferred/dropped, depending on time constraints.

Chapter 5. Background Research and Reading list

5.1. Existing Platform Gap Analysis

Only a few platforms currently provide digital support for new-build home personalisation in the UK. Housebuilder Pro offers a buyer-facing portal where developers upload product catalogues, and buyers browse choices and extras before submitting their selections [21]. Clixifix provides tools for managing customer care and defects, along with limited communication features before completion [22]. ContactBuilder helps developers manage buyer options through a CRM and extras portal, but it mainly supports selection rather than intelligent personalisation or procurement coordination [23]. The table below compares the publicly available features of selected platforms with the proposed scope of HomifyOne.

Features	Housebuilder Pro	Clixifix	ContactBuilder	HomifyOne
Buyer lifestyle preference capture	×	×	×	✓
AI-driven personalised product recommendations	×	×	×	✓
Conversational AI assistant	×	×	×	✓
Room-by-room selection workflow	✓	×	✓	✓
Real-time live budget tracker	×	×	×	✓
Product comparison tool	×	×	×	✓
Developer approval workflow	✓	×	✓	✓
Automated PO generation grouped by supplier	×	×	×	✓
Supplier fulfilment and invoice tracking	×	×	×	✓
Multi-role platform: Buyer, Developer, Supplier and Admin	×	×	×	✓
Real-time in-app messaging across stakeholders	×	✓	×	✓
PDF selection summary for buyers	×	×	×	✓

This comparison analysis confirms that no existing application combines AI-driven buyer preference intelligence, conversational guidance, procurement automation, and multi-stakeholder coordination in a single platform. Therefore, HomifyOne should be seen as a new type of tool, not just a better version of an existing one.

5.2. Research Element 1: NLP-Based Semantic Recommendation Engine

HomifyOne's recommendation engine is the technically novel part of the system. This solves a problem that most property platforms haven't fixed. People naturally describe their needs in everyday words, like wanting to have a home for a young family or one that's easy to maintain. Yet most catalogues still rely on rigid categories and filters. The recommendation engine is designed to bridge that gap by interpreting buyer language and matching it to relevant catalogue items.

Instead of relying only on manual filtering or static categories, the system converts both buyer questionnaire answers and product details into semantic representations, then compares them by similarity. Sentence-BERT is well suited to this. Reimers and Gurevych showed that the model produces dense sentence embeddings that perform well on semantic similarity tasks, which makes it a good fit for matching what a buyer wants to what a product actually offers [10]. To search across these embeddings quickly as the catalogue grows, the system uses FAISS (Facebook AI Similarity Search), based on the method developed by Johnson, Douze and Jégou for fast similarity search over large embedding spaces [11].

These technologies also suit HomifyOne for a more specific reason. As a new system, HomifyOne does not yet have the volume of buyer interaction data that collaborative filtering normally requires to work well. A semantic, knowledge-based approach is more realistic at this early stage. This matches the wider literature on hybrid recommender systems, which shows that combining several strategies, including content-based matching, rules and similarity scoring, tends to work better than relying on just one [12].

The benefit for buyers is easy to see in practice. A buyer who describes their household as a busy family with young children and a dog could be guided towards durable, easy-clean, scratch-resistant finishes rather than decorative upgrades that may not suit daily use. In this way, the catalogue stops being a passive list and becomes a tool that actively supports the buyer's decision, based on how they actually live.

5.3. Research Element 2: Conversational AI Assistant

The second research element is a conversational AI assistant that is grounded in HomifyOne's catalogue. Its purpose is to let buyers ask natural language questions, for example “What flooring suits a high-traffic hallway under £800?” or “Which kitchen upgrades are useful for a family home?”, without depending entirely on developer brochures or manual sales advice for answers that could otherwise be generated automatically.

This is made possible through Retrieval-Augmented Generation (RAG), a technique introduced by Lewis et al. It works by combining a generative language model with retrieved external context, so that what the model produces stays grounded in real source material rather than being invented [13]. Guu et al. reach a similar conclusion with REALM, showing that retrieval-augmented language models are valuable for tasks that depend heavily on accurate, specific knowledge [14]. This supports the same grounding approach used here. This grounding matters a great deal for HomifyOne. The assistant must not invent products, prices or specifications that are not in the live developer catalogue, because doing so could mislead a buyer making a significant financial decision. For this reason, the implementation will compare two approaches, one a retrieval-only pipeline built with FAISS which simply

returns ranked catalogue matches, and a full RAG pipeline which adds a Gemini-based conversational layer on top of retrieval.

Even with retrieval, hallucination can still occur if the generative model is not given proper instructions on how to use the retrieved content. Hence, Prompt engineering is required to control the assistant's behaviour, response and handle fallbacks. White et al. describe prompts as instructions that can enforce rules, automate processes and shape the quality of LLM output [19]. HomifyOne's Gemini prompt will guide the assistant to stay within the retrieved catalogue content, mention product names and prices when available, and avoid answering questions about products not found in the catalogue. This reflects Zamfirescu-Pereira et al.'s finding that LLMs work better when prompts are designed systematically instead of being written as vague instructions [20]. Therefore, the assistant will be designed as a catalogue-aware home personalisation advisor, helping to reduce hallucination risk and improve the reliability of its responses.

The final choice will however be based on four criteria:

1. Recommendation relevance
2. Response accuracy
3. Hallucination risk
4. Implementation feasibility

5.4. Theoretical Foundation for Lifestyle and Behavioural Preference Modelling

The lifestyle questionnaire in HomifyOne is based on established ideas from consumer behaviour and human-computer interaction. One important theory is lifestyle segmentation, mainly Plummer's Activities, Interests and Opinions model, which explains that consumer choices can be better understood by looking at how people live, what they value, and what practical needs influence their decisions [15]. This supports HomifyOne's use of buyer details such as household size, preferred interior style, room usage, storage needs, budget sensitivity, and lifestyle priorities.

This approach can also be related with Means-End Chain Theory which explains how consumers connect product attributes to practical benefits and personal values [16]. For example, durable flooring is not only a product choice, it can certainly represent easier maintenance, family practicality and long-term satisfaction. This supports HomifyOne's aim to recommend extras based on buyer preferences rather than simply displaying a static catalogue.

Furthermore, Choice Overload Theory justifies the need for AI-assisted filtering, as too many options can reduce decision confidence and satisfaction [17]. This is highly relevant to new-build personalisation, where buyers often face a large catalogue of upgrades within limited time and budget pressure. Finally, the Technology Acceptance Model also states that users are more likely to adopt a system when they feel it as useful and easy to use [18].

Therefore, HomifyOne's questionnaire, AI recommendation engine and simplified buyer journey are research-informed design decisions intended to improve confidence, usability and decision quality.

5.5. Reading List

1. HuggingFace, all-MiniLM-L6-v2 Model Card, <https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2> - Covers model documentation for the embedding model used in semantic search
2. Meta AI Research, FAISS Documentation, <https://faiss.ai> - Acts as an implementation guide for building the FAISS index
3. Google, Gemini API Documentation, <https://ai.google.dev/docs> - Useful API reference for integrating the conversational AI assistant
4. M. Richards, Software Architecture Patterns, O'Reilly Media, 2015 - This book contains everything about Event-driven architecture
5. Socket.io, Socket.io v4 Documentation, <https://socket.io/docs/v4> - Acts as an implementation guide for multi-room real-time messaging
6. OWASP Foundation, Authentication Cheat Sheet, <https://cheatsheetseries.owasp.org> - This article contains the security implementation standards for JWT and bcrypt
7. D. F. Ferraiolo and D. R. Kuhn, "Role-Based Access Controls," NIST-NCSC, 1992 - This journal provides some foundational theory for the four-role RBAC model
8. Express.js, Official Documentation, <https://expressjs.com> - Helpful for backend API and middleware implementation
9. MongoDB, Official Documentation, <https://www.mongodb.com/docs> - This documentation will be helpful for NoSQL database design for selections, products, users, and POs
10. Quadient, "The Transformative Impact of Purchase Order Automation," 2026 - This article offers the business context for automated PO generation design

Chapter 6. Time-plan and Risk Plan

6.1. Time-plan (Gantt chart)

The following Gantt chart shows a clear time plan of this project,

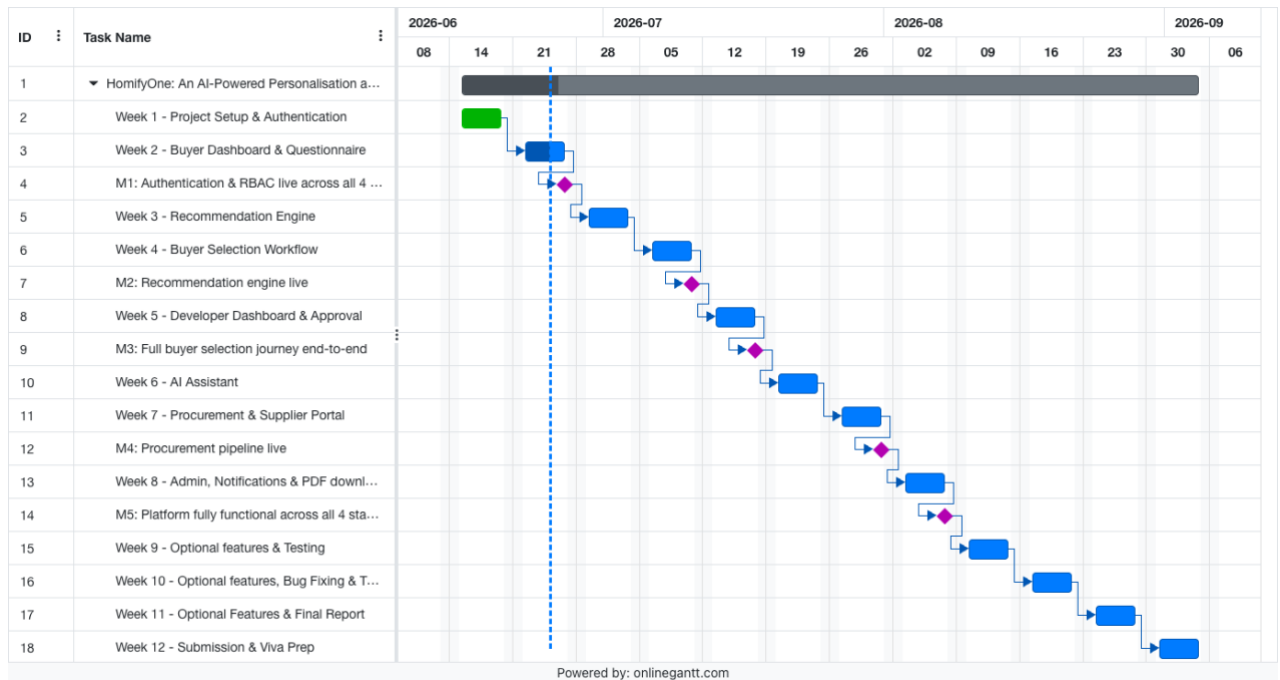


Figure 2: Project Timeline using Gantt chart

6.2. Detailed Timetable

S.No.	Task	Task Start Date	Task End Date
1.	Dev environment, MongoDB, backend setup	15/06/2026	16/06/2026
2.	JWT auth, bcrypt & RBAC (4 roles) API	17/06/2026	19/06/2026
3.	Login UI, role-based routing	19/06/2026	21/06/2026
4.	Auth API end-to-end testing (Postman)	22/06/2026	22/06/2026
5.	Buyer Dashboard	23/06/2026	24/06/2026
6.	Buyer Questionnaire	24/06/2026	26/06/2026

7.	Sentence-transformers embeddings for product catalogue	29/06/2026	01/07/2026
8.	Build FAISS index over product embeddings	01/07/2026	03/07/2026
9.	Recommendation API returning ranked products	03/07/2026	05/07/2026
10.	Recommendation results UI & budget tracker	06/07/2026	08/07/2026
11.	Room-by-room selection workflow	08/07/2026	10/07/2026
12.	Product comparison tool	10/07/2026	12/07/2026
13.	Developer CRUD & dashboard	13/07/2026	15/07/2026
14.	Developer approval / rejection workflow	15/07/2026	17/07/2026
15.	Status notifications to buyer on approval	17/07/2026	19/07/2026
16.	AI assistant feature	20/07/2026	26/07/2026
17.	Auto PO generation grouped by supplier	27/07/2026	30/07/2026
18.	Supplier portal	30/07/2026	02/08/2026
19.	Admin governance panel & user management	03/08/2026	05/08/2026
20.	Socket.io real-time notifications	05/08/2026	07/08/2026
21.	Real-time in-app messaging	06/08/2026	08/08/2026
22.	PDF selection summary download, Calendar sync features	08/08/2026	09/08/2026
23.	Unit testing, Integration testing	10/08/2026	12/08/2026
24.	End-to-end system testing	12/08/2026	13/08/2026
25.	Review the optional requirements and decide which requirement should be implemented next based on availability and project feasibility	12/08/2026	22/08/2026
26.	Final Report prep	20/08/2026	03/09/2026
27.	Final report submission & Viva prep	03/08/2026	04/08/2026

Note: The proposed timeline may change during development due to unforeseen challenges, bug resolution time,

or changes in project needs. Any revisions will be made carefully to maintain the quality and stability of the application.

6.3. Milestones

Date	Milestone	Description
15/06 – 26/06	Authentication & RBAC live across all 4 roles	JWT-secured login, bcrypt password hashing, and role-based access control are fully implemented and tested for Buyer, Developer, Supplier, and Admin. Each role routes to its own dashboard with correctly scoped data access.
29/06 – 10/07	Recommendation engine live	NLP based embedding pipeline and FAISS vector index are built over the product catalogue. The recommendation API returns ranked product results based on the buyer's lifestyle questionnaire responses.
13/07 – 17/07	Full buyer selection journey end-to-end	A buyer can complete the preference questionnaire, receive AI-ranked recommendations, make room-by-room selections with live budget tracking, submit to the developer, and receive an approval or rejection notification.
20/07 – 31/07	Procurement pipelines live	On developer approval, purchase orders are automatically generated and grouped by supplier. Suppliers can view, acknowledge, and update the status of assigned POs through the dedicated supplier portal.
03/08 – 07/08	Platform fully functional across all 4 stakeholder roles	All stakeholder dashboards, the AI assistant, real-time Socket.io notifications, in-app messaging and PDF selection summary are operational and tested end-to-end across all four roles.

6.4. Risk plan

- Since AI and NLP are new areas for me, dedicated time has been planned in Weeks 3 and 6 to research, prototype, and refine prompts before integrating them into the system. If the recommendations do not meet the expected quality, a fallback approach will be used to keep the project on track.
- If the Gemini API becomes unavailable or reaches its rate limit, the RAG-based assistant may not generate responses properly. To avoid this issue, a retrieval-only fallback will be implemented so the system can still show matched product information from FAISS.
- Since WebSocket connections can sometimes be unstable, real-time messages and notifications may not always be delivered consistently. To avoid this problem, the

dashboard will also show updates through REST-based polling. If Socket.io does not work reliably within the available time, the feature will degrade to polling-based notifications without disrupting the main application.

- Integrating the Google Calendar API for Buyer-Developer meetings and Supplier ETA synchronisation could be time-consuming due to OAuth 2.0 configuration. If this cannot be completed properly within the allocated time in Week 9, the calendar feature will be simplified into an in-app reminder system based on stored dates and notifications.
- The system will use both real-time in-app notifications through Socket.io and email notifications through Nodemailer as a fallback. This ensures that important workflow events are communicated to the right stakeholder even if one notification method fails.
- The database schema will be designed carefully at the start so that future modifications can be handled more easily.
- To prevent data loss, the project will maintain database backups and version control. MongoDB Atlas automated backups will be enabled so that the data can be recovered if a database failure occurs.

Chapter 7. References

- [1] The Housing Forum, *Building Homes Better: The Quality Challenge*. London, U.K.: The Housing Forum, Jun. 2023. [Online]. Available: <https://housingforum.org.uk/reports/key-publications/building-homes-better-the-quality-challenge>. [Accessed: Jun. 17, 2026].
- [2] Commission for Architecture and the Built Environment, *What Home Buyers Want: Attitudes and Decision Making Among Consumers*. London, U.K.: CABE, 2005. [Online]. Available: https://www.mae.co.uk/assets/pdfs/151123_Mae_Architects_What_Home_Buyers_Want_Ho_using_Architecture.pdf. [Accessed: Jun. 17, 2026].
- [3] Competition and Markets Authority, *Housebuilding Market Study: Final Report*. London, U.K.: CMA, Feb. 26, 2024. [Online]. Available: https://assets.publishing.service.gov.uk/media/65d8baed6efa83001ddcc5cd/Housebuilding_market_study_final_report.pdf. [Accessed: Jun. 17, 2026].
- [4] M. Schoenwitz, J. Gosling, M. Naim, and A. Potter, “How to build what buyers want: Unveiling customer preferences for prefabricated homes,” in *Proc. 29th Annu. ARCOM Conf.*, Reading, U.K., Sep. 2013, pp. 435–444.
- [5] EyeSite View, “How housebuilders can digitise the home buying journey,” *EyeSite View Blog*, Sep. 2025. [Online]. Available: <https://eyesiteview.co.uk/how-house-builders-can-digitise-the-home-buying-journey>. [Accessed: Jun. 17, 2026].
- [6] The Builders Daily, “2025: A turning point for homebuilding tech integration,” *The Builders Daily*, Feb. 2025. [Online]. Available: <https://www.thebuildersdaily.com/2025-a-turning-point-for-homebuilding-tech-integration>. [Accessed: Jun. 17, 2026].
- [7] M. A. Islam *et al.*, “Understanding high-involvement product purchase through an innovative machine learning approach: A case of housing type choice,” *Journal of Consumer Behaviour*, vol. 21, no. 6, pp. 1–18, 2022, doi: 10.1002/cb.2055.
- [8] X. Itämäki, “Streamlining the purchase order invoice process,” M.S. thesis, Tampere Univ. Applied Sciences, Tampere, Finland, 2025. [Online]. Available: <https://www.theseus.fi/handle/10024/885104>. [Accessed: Jun. 17, 2026].
- [9] ProcurePro, “The top 12 problems facing procurement in the construction sector,” *CPO Strategy*, Mar. 2025. [Online]. Available: <https://cpostrategy.media/blog/2025/03/04/the-top-12-problems-facing-procurement-in-the-construction-sector>. [Accessed: Jun. 17, 2026].
- [10] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence embeddings using Siamese BERT-networks,” in *Proc. EMNLP-IJCNLP*, 2019, pp. 3980–3990, doi: 10.18653/v1/D19-1410.
- [11] J. Johnson, M. Douze, and H. Jégou, “Billion-scale similarity search with GPUs,” *IEEE Transactions on Big Data*, vol. 7, no. 3, pp. 535–547, 2021, doi: 10.1109/TBDATA.2019.2921572.

- [12] R. Burke, “Hybrid recommender systems: Survey and experiments,” *User Modeling and User-Adapted Interaction*, vol. 12, no. 4, pp. 331–370, 2002, doi: 10.1023/A:1021240730564.
- [13] P. Lewis et al., “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in *Proc. Advances in Neural Information Processing Systems*, 2020, pp. 9459–9474. [Online]. Available: <https://proceedings.neurips.cc/paper/2020/hash/6b493230205f780e1bc26945df7481e5-Abstract.html>. [Accessed: Jun. 19, 2026].
- [14] K. Guu, K. Lee, Z. Tung, P. Pasupat, and M.-W. Chang, “REALM: Retrieval-augmented language model pre-training,” in *Proc. Int. Conf. Machine Learning*, 2020, pp. 3929–3938. [Online]. Available: <https://proceedings.mlr.press/v119/guu20a.html>. [Accessed: Jun. 19, 2026].
- [15] J. T. Plummer, “The concept and application of life style segmentation,” *Journal of Marketing*, vol. 38, no. 1, pp. 33–37, 1974, doi: 10.1177/002224297403800106.
- [16] J. Gutman, “A means-end chain model based on consumer categorization processes,” *Journal of Marketing*, vol. 46, no. 2, pp. 60–72, 1982, doi: 10.2307/3203341.
- [17] S. S. Iyengar and M. R. Lepper, “When choice is demotivating: Can one desire too much of a good thing?” *Journal of Personality and Social Psychology*, vol. 79, no. 6, pp. 995–1006, 2000, doi: 10.1037/0022-3514.79.6.995.
- [18] F. D. Davis, “Perceived usefulness, perceived ease of use, and user acceptance of information technology,” *MIS Quarterly*, vol. 13, no. 3, pp. 319–340, 1989, doi: 10.2307/249008.
- [19] J. White et al., “A prompt pattern catalog to enhance prompt engineering with ChatGPT,” arXiv preprint arXiv:2302.11382, 2023. [Online]. Available: <https://arxiv.org/abs/2302.11382>. [Accessed: Jun. 19, 2026].
- [20] J. D. Zamfirescu-Pereira, R. Y. Wong, B. Hartmann, and Q. Yang, “Why Johnny can’t prompt: How non-AI experts try and fail to design LLM prompts,” in *Proc. CHI Conf. Human Factors in Computing Systems*, 2023, doi: 10.1145/3544548.3581388.
- [21] Housebuilder Pro, “Buyer choices & extras: Upgrade management,” Housebuilder Pro. [Online]. Available: <https://housebuilderpro.co.uk/buyer-choices-and-extras/>. [Accessed: Jun. 20, 2026].
- [22] Clixifix, “Housebuilders,” Clixifix. [Online]. Available: <https://www.clixifix.com/our-solutions/housebuilders/>. [Accessed: Jun. 20, 2026].
- [23] ContactBuilder, “Housebuilder CRM software,” ContactBuilder. [Online]. Available: <https://www.contact-builder.co.uk/>. [Accessed: Jun. 20, 2026].